

MONGODB & NOSQL

Support de cours — Bases de données orientées documents,
modélisation et requêtes NoSQL

Niveau : Débutant / Intermédiaire

Prérequis : notions de bases de données, bases de la ligne de commande

8 pages — Document pédagogique

1. Introduction aux bases de données NoSQL

Les bases de données **NoSQL** ("Not Only SQL") sont des systèmes de gestion de données qui s'écartent du modèle relationnel classique (tables, jointures, schéma rigide) pour offrir plus de flexibilité, de scalabilité horizontale et de performance sur certains types de charges de travail.

Définition clé

Le NoSQL regroupe plusieurs familles de bases de données conçues pour stocker des données non structurées ou semi-structurées, souvent réparties sur plusieurs serveurs.

Les grandes familles NoSQL

- **Documents** — MongoDB, CouchDB — données au format JSON/BSON
- **Clé-valeur** — Redis, DynamoDB — paires clé/valeur simples et rapides
- **Colonnes larges** — Cassandra, HBase — optimisées pour de gros volumes
- **Graphes** — Neo4j — modélise des relations complexes entre entités

SQL vs NoSQL

Critère	SQL (relationnel)	NoSQL
Schéma	Fixe et structuré	Flexible / dynamique
Mise à l'échelle	Verticale (scale-up)	Horizontale (scale-out)
Relations	Jointures natives	Données imbriquées / dénormalisées
Cohérence	ACID stricte	Souvent cohérence à terme

2. Les concepts fondamentaux de MongoDB

MongoDB est la base de données orientée documents la plus utilisée. Elle stocke les données sous forme de documents **BSON** (format binaire proche de JSON), regroupés dans des collections.

Correspondance avec le monde relationnel

SQL	MongoDB
Base de données	Base de données
Table	Collection
Ligne (row)	Document
Colonne	Champ (field)
Clé primaire	_id

Exemple de document

```
{
  "_id": ObjectId("64f1a2b3c4d5e6f7a8b9c0d1"),
  "nom": "Dupont",
  "prenom": "Marie",
  "age": 29,
  "email": "marie.dupont@exemple.com",
  "adresse": {
    "ville": "Casablanca",
    "pays": "Maroc"
  },
  "competences": ["Python", "MongoDB", "Docker"]
}
```

Schéma flexible

Contrairement à une table SQL, deux documents d'une même collection peuvent avoir des champs différents. Cette flexibilité facilite l'évolution rapide des applications, au prix d'une discipline de modélisation à maintenir côté application.

3. Premiers pas : installation et opérations CRUD

MongoDB peut être installé localement, via Docker, ou utilisé en tant que service managé avec **MongoDB Atlas**. Les opérations de base suivent l'acronyme **CRUD** : Create, Read, Update, Delete.

Lancer MongoDB avec Docker

```
docker run -d --name mongo -p 27017:27017 mongo:7

mongosh "mongodb://localhost:27017"
```

Créer (Create)

```
db.utilisateurs.insertOne({
  nom: "Dupont",
  prenom: "Marie",
  age: 29
})

db.utilisateurs.insertMany([
  { nom: "Martin", age: 34 },
  { nom: "Bernard", age: 41 }
])
```

Lire (Read)

```
db.utilisateurs.find()
db.utilisateurs.find({ age: { $gt: 30 } })
db.utilisateurs.findOne({ nom: "Dupont" })
```

Modifier et supprimer

```
db.utilisateurs.updateOne(
  { nom: "Dupont" },
  { $set: { age: 30 } }
)

db.utilisateurs.deleteOne({ nom: "Bernard" })
```

4. Requêtes et filtres avancés

MongoDB propose un riche langage de requêtes basé sur des opérateurs préfixés par \$, permettant des filtres précis et des mises à jour ciblées.

Opérateurs de comparaison

- **\$eq** / **\$ne** — égal / différent de
- **\$gt** / **\$gte** — supérieur à / supérieur ou égal à
- **\$lt** / **\$lte** — inférieur à / inférieur ou égal à
- **\$in** / **\$nin** — valeur présente / absente dans une liste

Opérateurs logiques

```
db.utilisateurs.find({
  $and: [
    { age: { $gte: 25 } },
    { age: { $lte: 40 } }
  ]
})

db.utilisateurs.find({
  $or: [ { ville: "Casablanca" }, { ville: "Rabat" } ]
})
```

Projection, tri et pagination

```
db.utilisateurs.find(
  { age: { $gte: 25 } },
  { nom: 1, email: 1, _id: 0 }
).sort({ age: -1 }).skip(10).limit(5)
```

Index

Les **index** accélèrent considérablement les requêtes en évitant un parcours complet de la collection (collection scan).

```
db.utilisateurs.createIndex({ email: 1 }, { unique: true })
```

5. Modélisation des données dans MongoDB

La modélisation NoSQL diffère fondamentalement de la normalisation SQL : l'objectif principal est d'optimiser les schémas pour les requêtes les plus fréquentes de l'application, quitte à dupliquer certaines données.

Embedding (documents imbriqués)

On imbrique les données liées directement dans le document parent, ce qui est efficace lorsque les données sont consultées ensemble et ne grossissent pas indéfiniment.

```
{
  "_id": 1,
  "titre": "Introduction à MongoDB",
  "auteur": "Claude",
  "commentaires": [
    { "utilisateur": "Alice", "texte": "Très clair !" },
    { "utilisateur": "Bob", "texte": "Merci pour le cours" }
  ]
}
```

Référencement (relations entre collections)

Pour des données volumineuses, partagées ou fréquemment mises à jour indépendamment, on préfère stocker une référence (ObjectId) vers un autre document, à la manière d'une clé étrangère.

```
// Collection "commandes"
{ "_id": 101, "client_id": ObjectId("64f1..."), "total": 59.90 }

// Récupération avec agrégation (lookup)
db.commandes.aggregate([
  { $lookup: {
    from: "clients",
    localField: "client_id",
    foreignField: "_id",
    as: "client"
  }}
])
```

Règle générale

Embedding pour les données consultées ensemble et de taille bornée ; référencement pour les données volumineuses, partagées entre plusieurs documents ou mises à jour fréquemment.

6. Le pipeline d'agrégation

Le **pipeline d'agrégation** permet de transformer et d'analyser les données en enchaînant plusieurs étapes de traitement, à la manière d'un pipeline Unix ou d'une requête SQL complexe (GROUP BY, JOIN, etc.).

Étapes courantes

- **\$match** — filtre les documents (équivalent de WHERE)
- **\$group** — regroupe et calcule des agrégats (équivalent de GROUP BY)
- **\$sort** — trie les résultats
- **\$project** — sélectionne ou transforme les champs à afficher
- **\$lookup** — effectue une jointure avec une autre collection

Exemple : chiffre d'affaires par ville

```
db.commandes.aggregate([
  { $match: { statut: "payee" } },
  { $group: {
    _id: "$ville",
    totalVentes: { $sum: "$montant" },
    nombreCommandes: { $sum: 1 }
  }},
  { $sort: { totalVentes: -1 } }
])
```

Résultat obtenu

```
[
  { "_id": "Casablanca", "totalVentes": 15400, "nombreCommandes": 210 },
  { "_id": "Rabat",      "totalVentes": 9800,  "nombreCommandes": 145 }
]
```

7. Scalabilité, réplication et intégration applicative

Réplication (Replica Set)

Un **replica set** est un groupe de serveurs MongoDB maintenant les mêmes données. Un nœud **primaire** reçoit les écritures, tandis que les nœuds **secondaires** répliquent les données et prennent le relais automatiquement en cas de panne (failover).

Partitionnement (Sharding)

Le **sharding** répartit les données sur plusieurs serveurs (shards) selon une clé de partitionnement (shard key), permettant de faire évoluer horizontalement des volumes de données très importants.

Utiliser MongoDB depuis une application

MongoDB propose des pilotes (drivers) officiels pour la plupart des langages. Exemple avec Python et PyMongo :

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017")
db = client["boutique"]

db.produits.insert_one({"nom": "Clavier", "prix": 29.99})

for produit in db.produits.find({"prix": {"$lt": 50}}):
    print(produit)
```

À retenir

La réplication assure la disponibilité des données ; le sharding assure la scalabilité horizontale. Les deux mécanismes peuvent être combinés en production.

8. Bonnes pratiques et conclusion

Bonnes pratiques de modélisation

- Concevoir le schéma en fonction des requêtes de l'application, pas l'inverse
- Éviter les documents illimités en taille (limite de 16 Mo par document)
- Créer des index sur les champs fréquemment filtrés ou triés
- Utiliser la validation de schéma (JSON Schema) pour garder une structure cohérente
- Surveiller les performances avec explain() sur les requêtes critiques

Validation de schéma

```
db.createCollection("utilisateurs", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["nom", "email"],
      properties: {
        email: { bsonType: "string", pattern: "^.+@.+$" }
      }
    }
  }
})
```

Points clés à retenir

- Le NoSQL regroupe plusieurs familles adaptées à des besoins différents
- MongoDB stocke des documents JSON/BSON flexibles dans des collections
- Le langage de requêtes couvre le CRUD, les filtres et l'agrégation
- La modélisation privilégie les requêtes de l'application (embedding vs référencement)
- Réplication et sharding assurent disponibilité et scalabilité horizontale

Pour aller plus loin

- Documentation officielle MongoDB (mongodb.com/docs)
- MongoDB Atlas : hébergement managé dans le cloud
- Recherche textuelle et vectorielle avec Atlas Search
- Comparaison approfondie avec d'autres bases NoSQL (Redis, Cassandra)

Conclusion

MongoDB illustre bien la philosophie NoSQL : flexibilité du schéma, scalabilité horizontale et modélisation orientée vers les besoins de l'application. Elle complète, plutôt qu'elle ne remplace systématiquement, les bases de données relationnelles selon les cas d'usage.